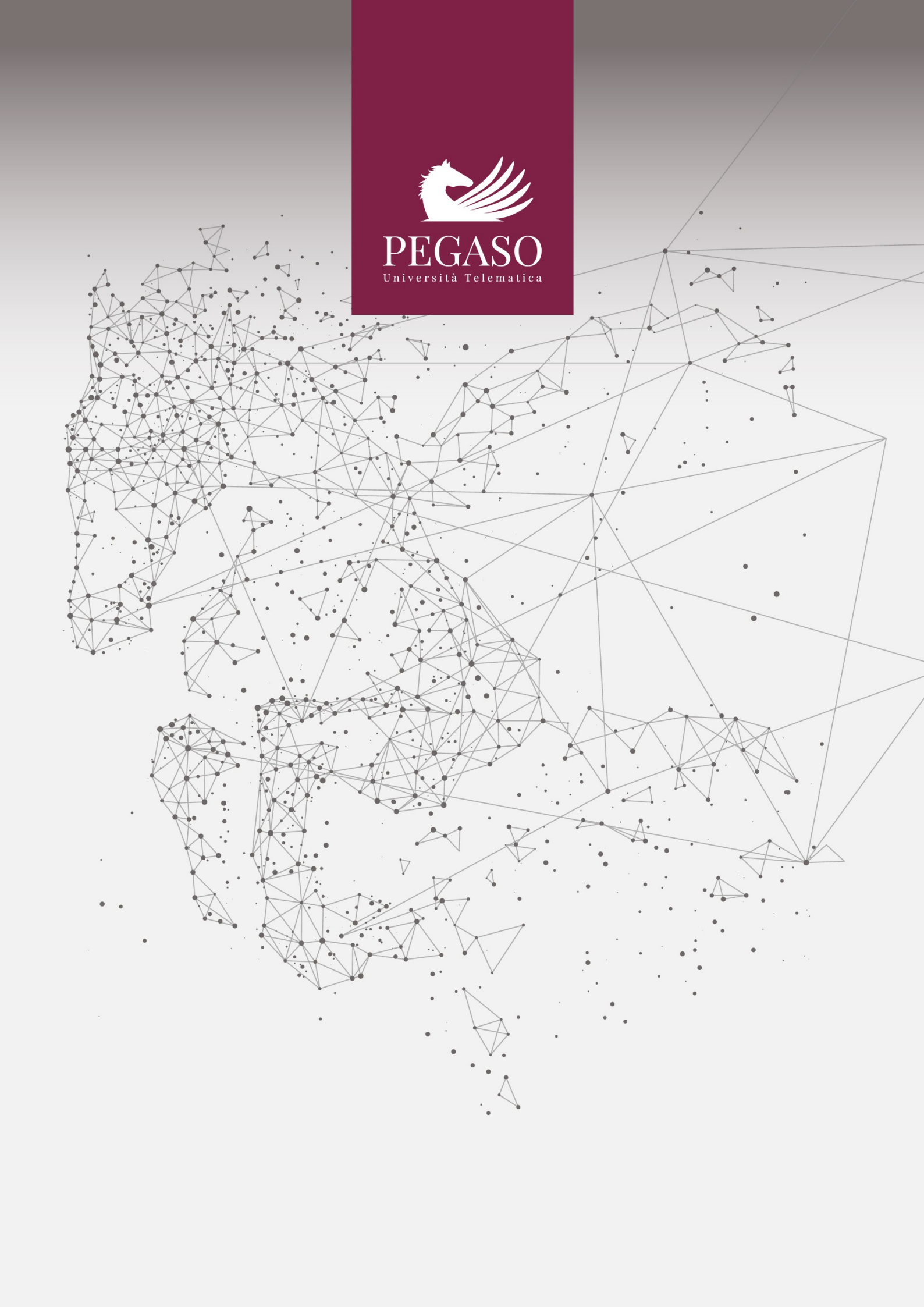

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. DATABASE NOSQL 3
2. MONGODB 5
3. MONGODB & NODEJS 11
BIBLIOGRAFIA 16
SITOGRAFIA 17

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Database NoSQL

I database SQL (Structured Query Language) sono stati per decenni la soluzione predominante per la gestione dei dati strutturati. Con un approccio tabellare, i database SQL utilizzano un insieme di relazioni tra tabelle per organizzare e interrogare i dati. Tuttavia, con l'esplosione dei dati non strutturati, come i documenti JSON, i database relazionali SQL si sono rivelati limitati.

I database NoSQL sono un insieme di database non relazionali che si differenziano dai tradizionali database SQL in vari modi: in generale, i database NoSQL sono progettati per gestire grandi quantità di dati non strutturati o semistrutturati, come ad esempio i dati in formato JSON.

Una delle principali caratteristiche dei database NoSQL è la loro capacità di gestire dati di qualsiasi formato, senza bisogno di una struttura rigida predefinita. Ciò significa che i dati possono essere memorizzati in modo flessibile e scalabile, senza la necessità di definire uno schema rigido a priori. Inoltre, i database NoSQL sono spesso progettati per scalare orizzontalmente su più server, in modo da gestire carichi di lavoro sempre più grandi.

I dati nei database NoSQL sono generalmente memorizzati in modo diverso rispetto ai database relazionali SQL:

- molti database NoSQL utilizzano un modello a **chiave-valore**, dove ogni valore è un blob di dati che può essere di qualsiasi formato
- altri database NoSQL utilizzano un modello basato su **documenti**, dove ogni documento rappresenta un'entità specifica e contiene tutti i dati correlati
- altri ancora utilizzano un **modello a grafo**, in cui i dati sono organizzati come nodi e archi, e possono essere interrogati utilizzando algoritmi di ricerca basati su grafi
- infine, alcuni database NoSQL utilizzano un **modello di colonne**, dove i dati vengono organizzati per colonna anziché per riga, in modo da consentire una memorizzazione più efficiente e una migliore scalabilità.

In questo contesto entra in gioco **MongoDB**, un database NoSQL progettato per gestire grandi quantità di dati non strutturati. MongoDB è un database NoSQL orientato ai documenti, progettato per gestire grandi quantità di dati non strutturati o semistrutturati, come ad esempio i dati in formato JSON. I dati in MongoDB sono organizzati in **collezioni**, dove ogni collezione rappresenta un gruppo di **documenti** correlati.

I documenti in MongoDB sono simili a record JSON, composti da un insieme di campi che contengono i dati. I campi possono contenere qualsiasi tipo di dato, inclusi array e sotto-documenti.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

A differenza dei database SQL tradizionali, non è necessario definire uno schema rigido a priori per ogni collezione in MongoDB, poiché i documenti possono essere memorizzati in modo flessibile e scalabile.

MongoDB offre anche funzionalità avanzate come la scalabilità orizzontale, l'indicizzazione flessibile e le query ad alte prestazioni. In particolare, MongoDB utilizza una tecnica chiamata sharding per distribuire i dati su più server, consentendo una scalabilità orizzontale su un numero elevato di macchine. Inoltre, MongoDB supporta l'indicizzazione flessibile, consentendo agli utenti di indicizzare i dati in base a qualsiasi campo nei documenti. Ciò consente di eseguire ricerche rapide e performanti su grandi quantità di dati.

Le query in MongoDB sono scritte utilizzando il linguaggio di query di MongoDB, che utilizza un modello di query basato su documenti: ciò significa che le query possono essere eseguite sui campi dei documenti, inclusi i campi annidati e gli array; MongoDB offre anche funzionalità avanzate come l'aggregazione, che consente di combinare e analizzare i dati su più documenti.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. MongoDB

Per creare un database MongoDB su MongoDB Atlas, è possibile seguire i seguenti passaggi:

- **Creare un account su MongoDB Atlas**: l'utente può creare un account su MongoDB Atlas seguendo le istruzioni fornite sulla pagina di registrazione https://www.mongodb.com/cloud/atlas
- **Creare un nuovo cluster**: dopo aver effettuato l'accesso a MongoDB Atlas, l'utente può creare un nuovo cluster seguendo le istruzioni sulla dashboard; l'utente può scegliere le opzioni di configurazione del cluster, come il provider di cloud, la regione e la dimensione del cluster
- **Aggiungere un utente al database**: l'utente può creare un nuovo utente per il database seguendo le istruzioni sulla dashboard; l'utente può assegnare un nome utente e una password per l'utente e assegnare le autorizzazioni necessarie per accedere al database
- **Configurare le impostazioni di rete**: l'utente può configurare le impostazioni di rete per consentire la connessione al database dal proprio server; l'utente può aggiungere l'indirizzo IP del proprio server o utilizzare "0.0.0.0/0" per consentire l'accesso da qualsiasi indirizzo IP
- **Connettere il server al database**: l'utente può utilizzare la stringa di connessione fornita sulla dashboard per connettersi al database dal proprio server; l'utente può utilizzare un driver MongoDB compatibile con il proprio linguaggio di programmazione preferito per connettersi al database
- **Aggiungere dati al database**: l'utente può utilizzare il driver MongoDB per aggiungere dati al database, come descritto nella documentazione del driver MongoDB per il proprio linguaggio di programmazione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Creazione di un nuovo progetto:

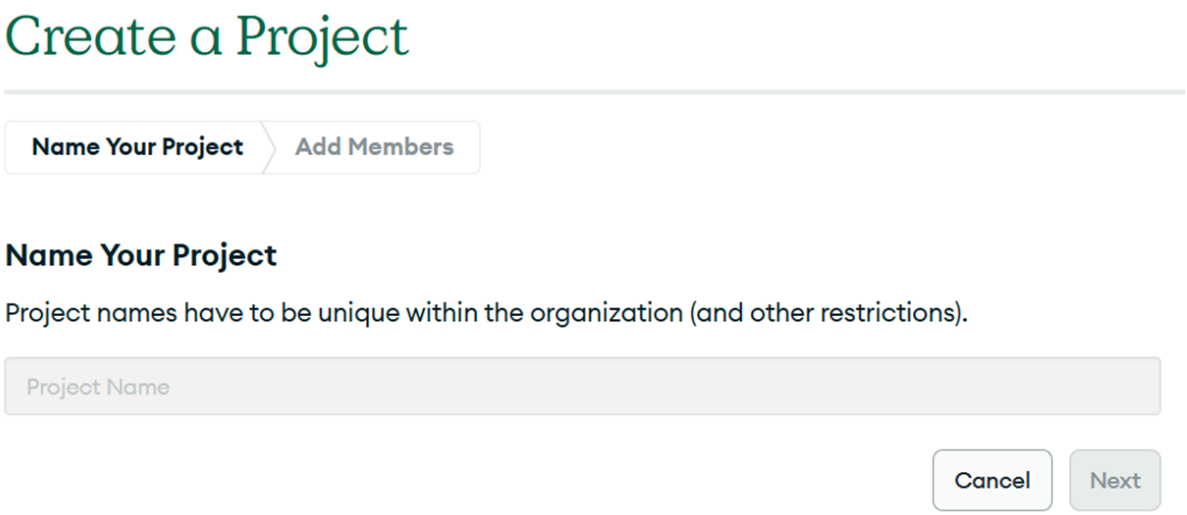


Figura 1 *- Creazione nuovo progetto*

Creazione del database:

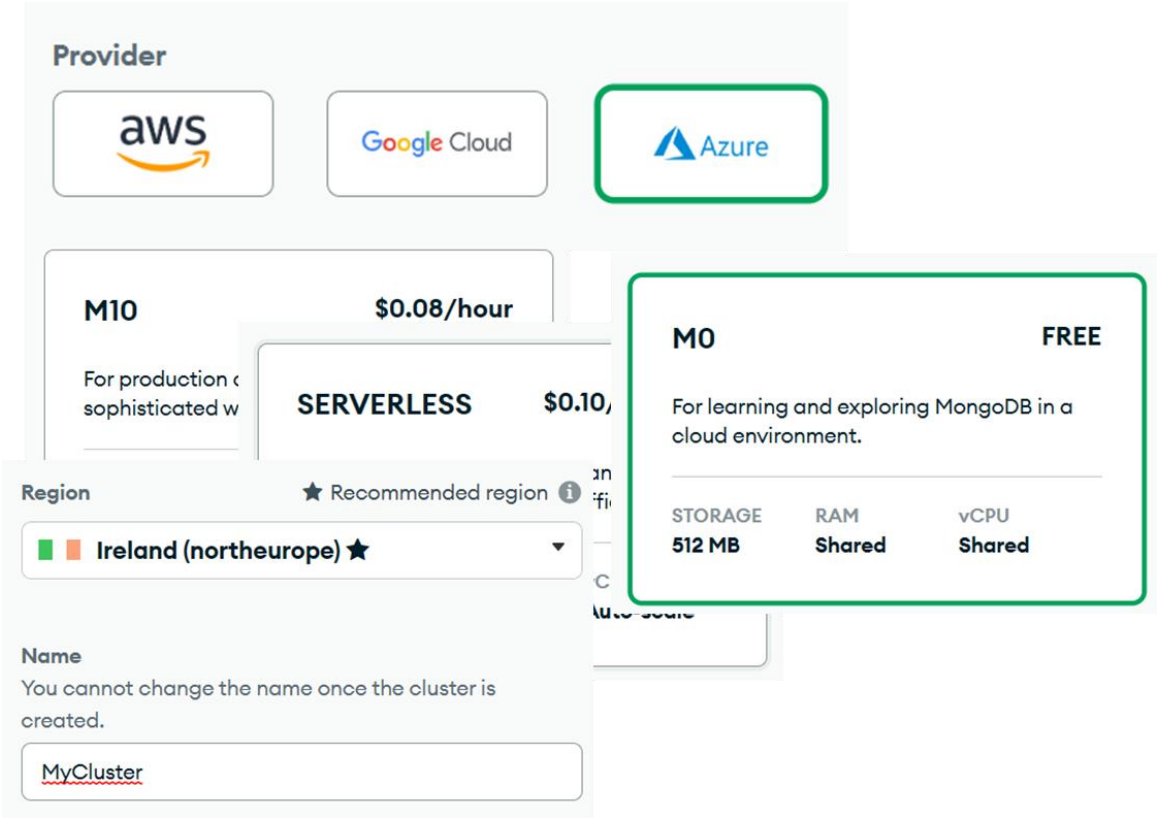


Figura 2 *- Creazione del database*

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Scelta dell'ambiente di deployment:

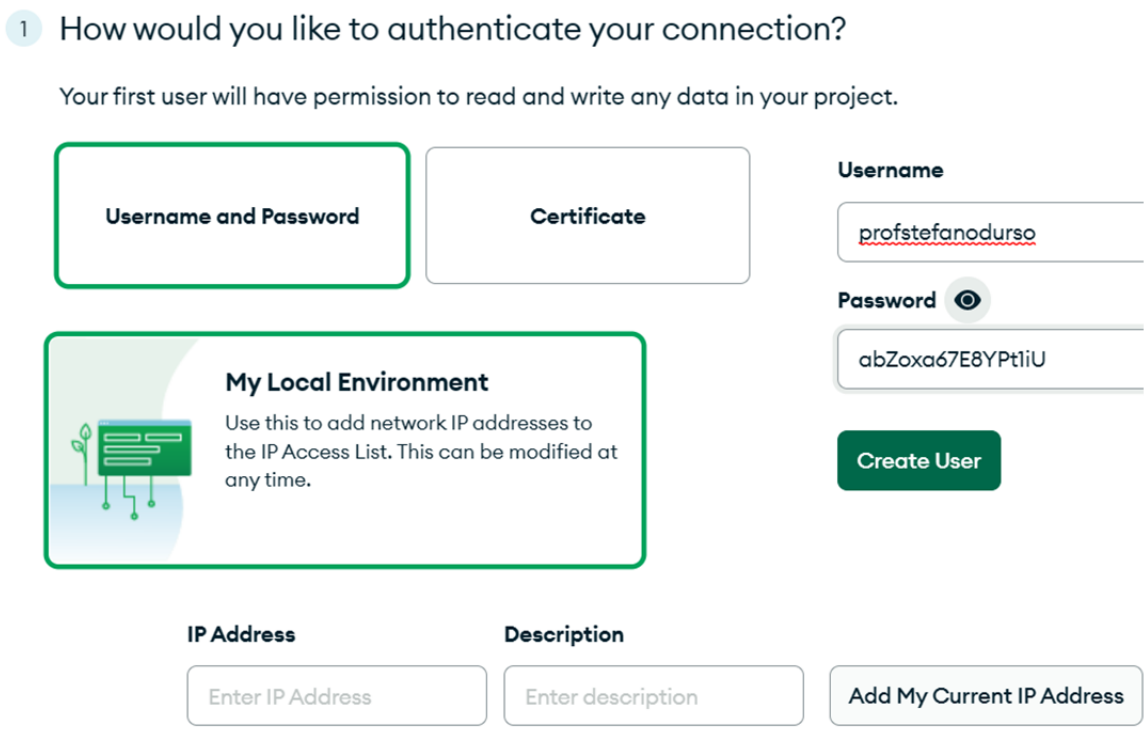


Figura 3 *- Ambiente di deployment*

È ora possibile specificare le credenziali di accesso e la possibilità di connettersi da uno specifico indirizzo IP (questa informazione può essere specificata anche successivamente):

A questo punto è possibile collegarsi al nostro database scegliendo le modalità proposte.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

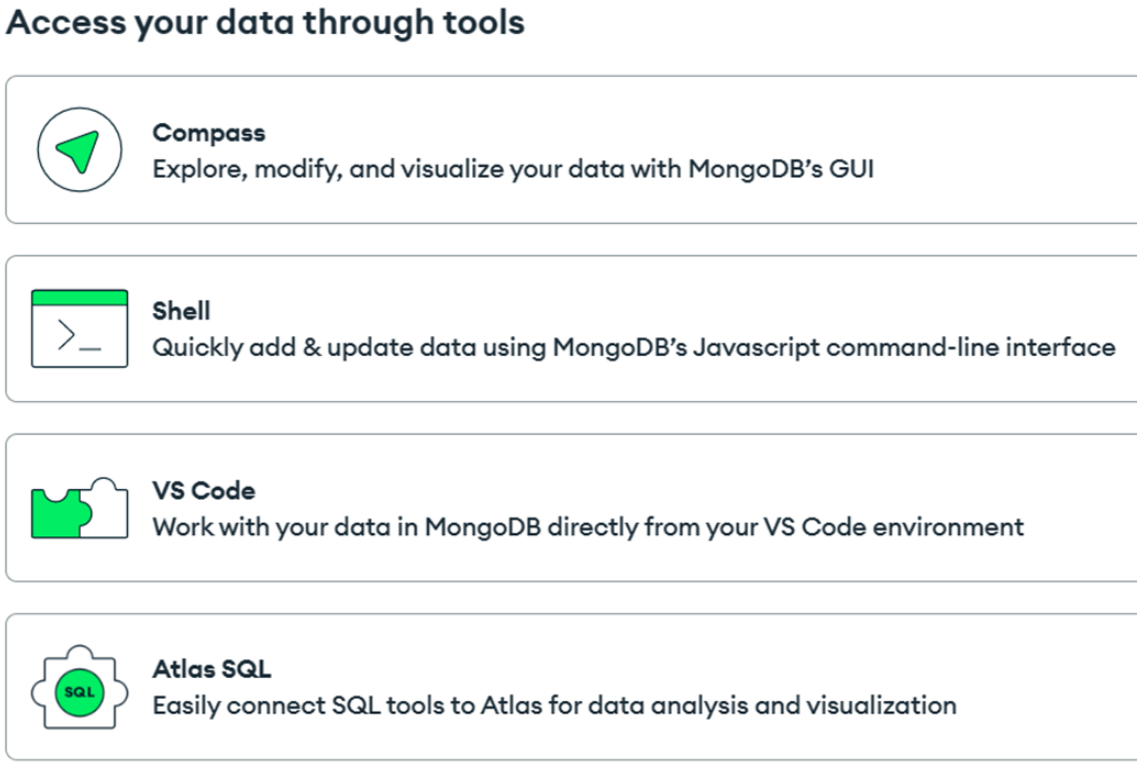


Figura 4 *- Connessione a MongoDB*

È possibile scaricare MongoDB shell e seguire le indicazioni di seguito:

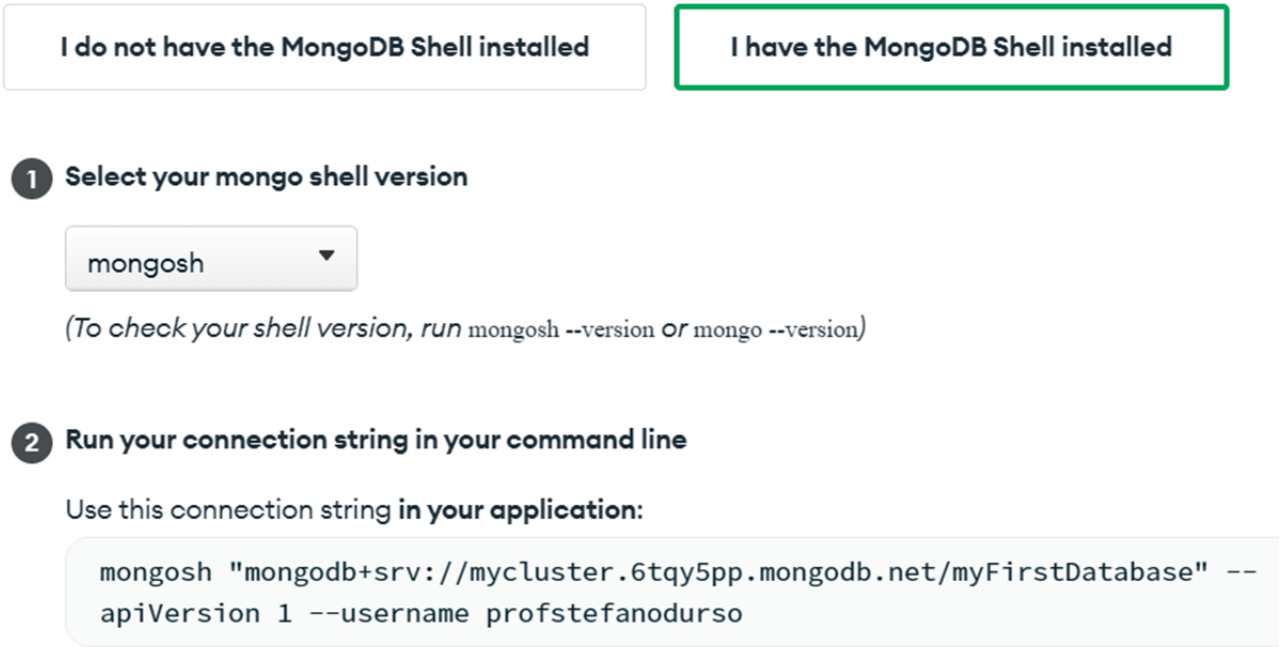


Figura 5 *- MongoDB shell*

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ecco alcuni comandi utili per lavorare con MongoDB attraverso la MongoDB shell:

- **show dbs**: visualizza una lista dei database disponibili;
- **use <nome_database>**: seleziona il database su cui si desidera lavorare;
- **db.createCollection(<nome_collezione>)**: creazione di una nuova collezione in un database specifico;
- **db.<nome_collezione>.drop()**: eliminazione di una collezione esistente in un database specifico
- **show collections**: visualizza una lista delle collections disponibili;
- **db.<nome_collezione>.find()**: visualizza tutti i documenti nella collezione specificata;
- **db.<nome_collezione>.insertOne(<documento>)**: aggiunge un nuovo documento alla collezione specificata;
- **db.<nome_collezione>.insertMany(<documenti>)**: aggiunge più documenti alla collezione specificata;
- **db.<nome_collezione>.updateOne(<filtro>, <aggiornamento>)**: aggiorna il primo documento che soddisfa il filtro specificato nella collezione specificata;
- **db.<nome_collezione>.updateMany(<filtro>, <aggiornamento>)**: aggiorna tutti i documenti che soddisfano il filtro specificato nella collezione specificata;
- **db.<nome_collezione>.deleteOne(<filtro>)**: elimina il primo documento che soddisfa il filtro specificato nella collezione specificata;
- **db.<nome_collezione>.deleteMany(<filtro>)**: elimina tutti i documenti che soddisfano il filtro specificato nella collezione specificata.

Una collezione è l'equivalente di una tabella in un database relazionale. Una collezione è un raggruppamento logico di documenti in MongoDB, dove ogni documento rappresenta un singolo record all'interno della collezione. Le collezioni in MongoDB sono schemaless, il che significa che ogni documento all'interno di una collezione può avere un diverso schema dei campi: ciò differisce dalle tabelle in un database relazionale, dove ogni riga della tabella ha gli stessi campi definiti dalla struttura della tabella.

In MongoDB, è possibile eseguire operazioni di query e di modifica sui documenti all'interno delle collezioni utilizzando la MongoDB Query Language (MQL) e il MongoDB Aggregation Framework: questi strumenti consentono di gestire i dati in modo flessibile e di adattarsi ai cambiamenti dello schema dei dati nel tempo.

In generale, la struttura flessibile delle collezioni in MongoDB consente di gestire dati complessi in modo più efficiente rispetto a un database relazionale tradizionale.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Eseguiamo alcune operazioni sul nostro database:

```
use mydb
db.createCollection("users")
db.users.insertMany([
  { name: "Paolo Bianchi", email: "paolo.bianchi@example.com" },
  { name: "Giulia Verdi", email: "giulia.verdi@example.com" },
  { name: "Luca Neri", email: "luca.neri@example.com" }
])
db.users.find()
db.users.insertOne({name:"Mario Red",email: "mario.red@ex.com"})
db.users.find()
db.users.drop()
```

Ecco una breve descrizione di ciascuna istruzione:

- **use mydb**: questo comando seleziona il database "mydb" come database corrente.
- **db.createCollection("users")**: questo comando crea una nuova collezione "users" all'interno del database corrente.
- **db.users.insertMany([...])**: questo comando inserisce tre documenti nella collezione "users" contenenti informazioni sugli utenti. Ogni documento ha un campo "name" e un campo "email" impostati con valori specifici.
- **db.users.find()**: questo comando restituisce tutti i documenti nella collezione "users".
- **db.users.insertOne({...})**: questo comando inserisce un nuovo documento nella collezione "users" con i campi "name" e "email" impostati ai valori specificati.
- **db.users.find()**: questo comando restituisce tutti i documenti nella collezione "users", compreso il nuovo documento inserito al punto precedente.
- **db.users.drop()**: questo comando elimina la collezione "users" e tutti i documenti al suo interno.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. MongoDB & Nodejs

Se volessi collegare un'applicazione sviluppata in node al mio mongoDB dovrei innanzitutto autorizzare l'ip del mio server. Se stiamo operando su replit è possibile accedere alla shell e digitare:

```
curl ifconfig.me
```

Nella sezione relativa al Network Access è possibile inserire l'IP individuato:

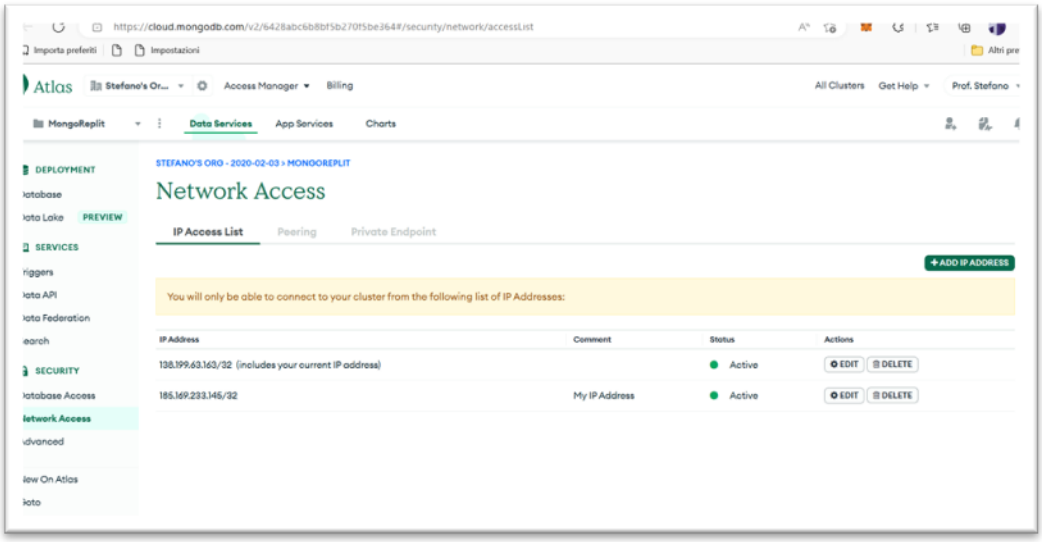


Figura 6 *- Network Access*

È ora necessario configurare la stringa di connessione che la nostra applicazione dovrà utilizzare.

ATTENZIONE: usare come stringa di connessione la struttura fornita scegliendo "Connect using VScode"

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

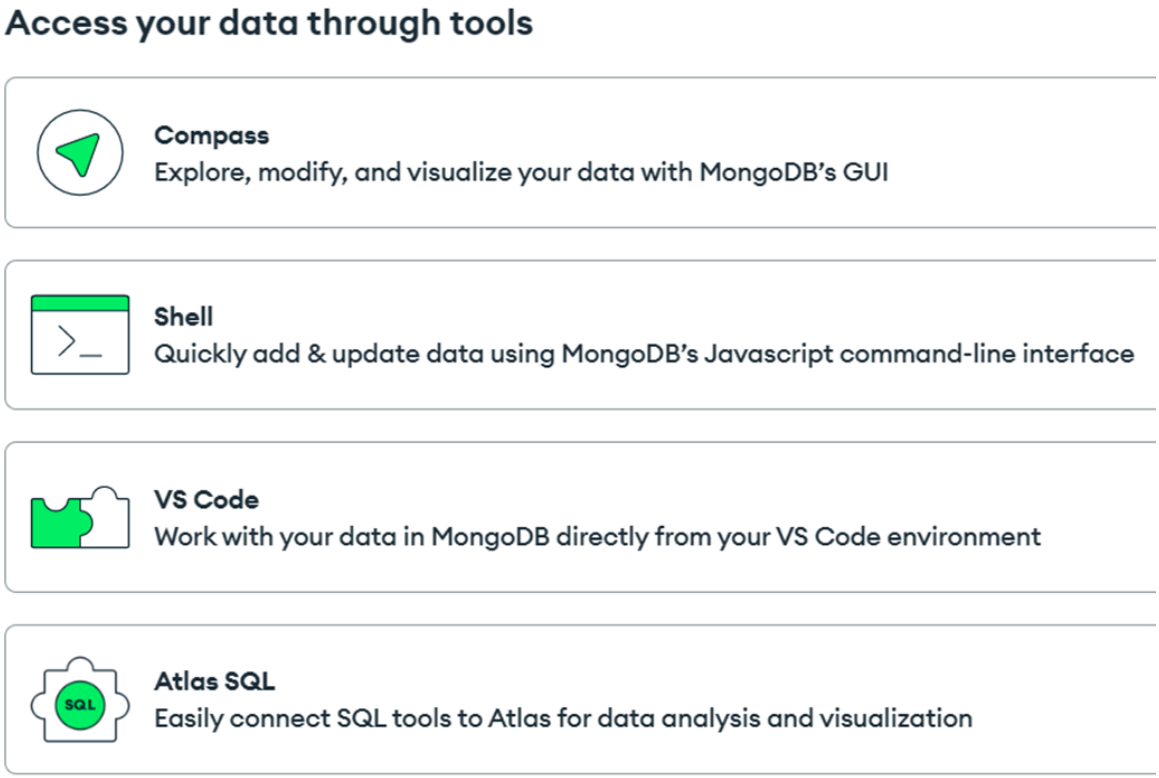


***Figura 7** - Connecti using VS Code*

E successivamente selezionare la string contenuta in "Connect to your MongoDB Palette":

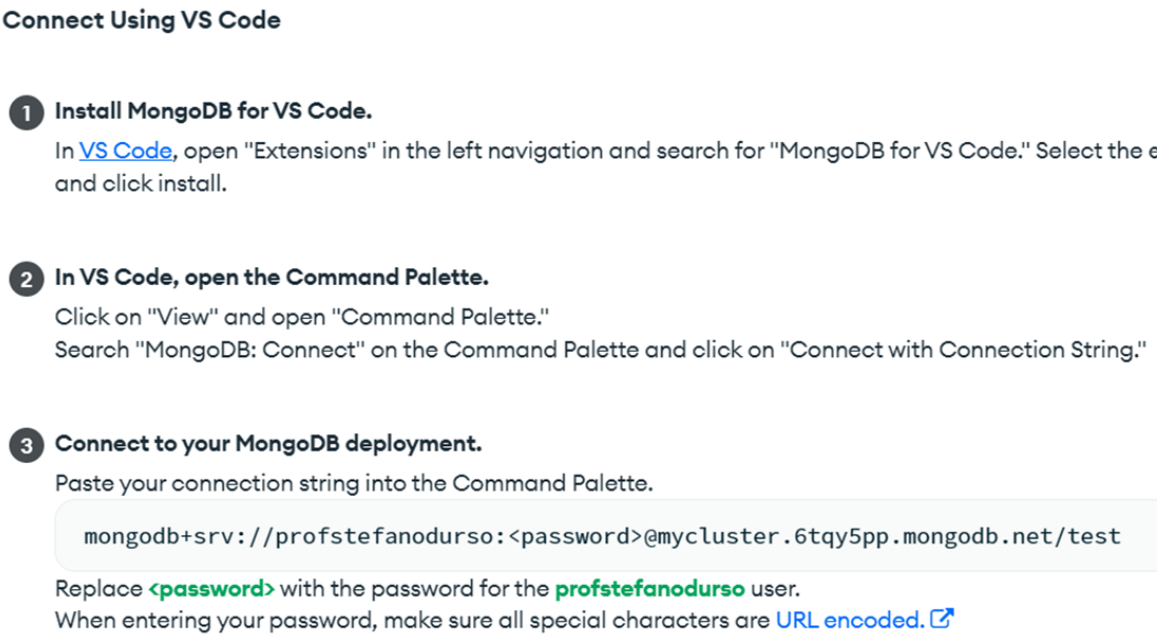


***Figura 8** - Connect to your MongoDB deployment*

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La stringa è nella forma generica:

```
mongodb+srv://<username>:<password>@mydb.twv6equ.mongodb.net/<collection>
```

Occorre sostituire i singoli campi <username>, <password> e <collection> con i valori relativi ai nostri parametri.

A questo punto possiamo configurare la nostra applicazione nodejs:

```
const mongoose = require('mongoose');

async function connect() {
  try {
    await
mongoose.connect('mongodb+srv://<username>:<pwd>@mydb.twv6equ.mongodb.net/<coll
ection>', {
      useNewUrlParser: true,
      useUnifiedTopology: true
    });
    console.log('Connected to MongoDB');
  } catch (error) {
    console.error(error);
  }
}

async function listCollections() {
  try {
    const collections = await
            mongoose.connection.db.listCollections().toArray();
    console.log(collections);
  } catch (error) {
    console.error(error);
  } finally {
    mongoose.disconnect();
  }
}

async function getFirstCollection() {
    try {
      const collections = await
            mongoose.connection.db.listCollections().toArray();
      const firstCollection = collections[0];
      console.log(firstCollection.name)
      const Model =
            mongoose.model(
                  firstCollection.name,
                  new mongoose.Schema({}),firstCollection.name);
      const documents = await Model.find({});
      console.log(documents);
    } catch (error) {
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
      console.error(error);
    } finally {
      mongoose.disconnect();
    }
}

async function searchDocument() {
    try {
      const Model = mongoose.model(
            'User', new mongoose.Schema({name: String}), 'users');
      const document = await Model.findOne({ name: 'Paolo Bianchi' });
      console.log(document);
    } catch (error) {
      console.error(error);
    } finally {
      mongoose.disconnect();
    }
}

async function searchDocuments() {
    try {
      const Model = mongoose.model('User', new mongoose.Schema({
        name: String}), 'users');
      const document = await Model.find({});
      console.log(document);
    } catch (error) {
      console.error(error);
    } finally {
      mongoose.disconnect();
    }
}

connect()
  //.then(() => listCollections())
  //.then(() => getFirstCollection())
  //.then(() => searchDocument())
  .then(() => searchDocuments())
  .catch((error) => console.error(error));
```

Di seguito un commento del codice:

- Viene importata la libreria Mongoose.
- Viene definita una funzione connect()asincrona che si connette al database MongoDB utilizzando l'URL della connessione specificata; viene impostata la configurazione useNewUrlParser e useUnifiedTopology per garantire che la connessione venga stabilita correttamente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- Viene definita una funzione listCollections() asincrona che utilizza il metodo listCollections() per elencare tutte le collezioni presenti nel database e stamparle in console.
- Viene definita una funzione getFirstCollection() asincrona che utilizza il metodo listCollections() per ottenere il nome della prima collezione presente nel database e quindi utilizza il nome della collezione per creare un nuovo modello Mongoose vuoto. Viene quindi utilizzato il metodo find() per recuperare tutti i documenti nella collezione e stamparli in console.
- Viene definita una funzione searchDocument() asincrona che crea un modello Mongoose per la collezione "users" con il campo "name" di tipo stringa. Viene quindi utilizzato il metodo findOne() per cercare un documento nella collezione in base al valore del campo "name" e stamparlo in console.
- Viene definita una funzione searchDocuments() asincrona che crea un modello Mongoose per la collezione "users" con il campo "name" di tipo stringa. Viene quindi utilizzato il metodo find() per cercare tutti i documenti nella collezione e stamparli in console.
- Viene chiamata la funzione"connect() per connettersi al database.
- Viene chiamata una delle funzioni listCollections(), getFirstCollection(), searchDocument() o searchDocuments(), a seconda del commento presente sulla riga corrispondente. Tutte le funzioni restituiscono una Promise che viene gestita dal metodo then() e catch().

La sintassi di mongoose.model() è la seguente:

```
mongoose.model(name, schema, collection, skipInit)
```

- name è il nome del modello
- schema è lo schema Mongoose utilizzato per definire la struttura dei documenti della collezione
- collection è il nome della collezione associata al modello nel database
- skipInit è un parametro booleano facoltativo che indica se eseguire o meno l'inizializzazione automatica del modello (valore predefinito: false).

In generale, questa funzione viene utilizzata per creare un nuovo modello Mongoose che rappresenta una collezione in un database MongoDB. Il modello consente di effettuare operazioni come la ricerca, l'aggiunta, la modifica o la cancellazione di documenti nella collezione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010.
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- Marco Andreini, Patrizia Andronico, Arianna Del Soldato, Cristian Lucchesi, Maurizio Martinelli, Giuseppe Vasarelli. Manuale corso di Tecnologie Web Istituto di Informatica e Telematica del CNR, 2005.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Sitografia

- HTML 4.01 Specification: http://www.w3.org/TR/html401/
- HTML5: http://www.w3.org/TR/html5/
- Javascript: JavaScript Tutorial (w3schools.com)
- HTML: http://www.w3schools.com/html/default.asp
- CSS: https://www.w3schools.com/css/default.asp
- CSS: http://www.w3.org/TR/CSS/
- NODEJS: https://nodejs.org

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).